

作业10 & 期末复习1 时间片轮转调度，sleep系统调用和盘交换区的使用

同济大学计算机系操作系统 国豪2023 拔尖班2023

姓名 毛星博 学号 2351520

Part 1、Unix V6 时间片轮转调度的实现

习题： Unix V6++系统中存在 3 个 CPU bound 用户态进程 PA、PB 和 PC。3 个进程静态优先数相等：100，p_cpu 是 0。Process[8]、[5]、[9]分别是 PA 、PB、PC 进程的 PCB。T 时刻是整数秒，PA 先运行。

1、画进程运行时序图。

初始状态：T 时刻，静态优先数均为 100，p_cpu 均为 0。

调度参数：HZ = 60（一秒 60 个滴答），SCHMAG = 20（每秒衰减值）。

优先级计算公式：p_pri = 100 + p_cpu / 16。

p_cpu 表

	T	T+1	T+2	T+3	T+4
PA	0	40	20	0	40
PB	0	0	40	20	0
PC	0	0	0	40	20

p_pri 表

	T	T+1	T+2	T+3	T+4
运行进程	PA	PB	PC	PA	PB
PA	100	102	101	100	102
PB	100	100	102	101	100
PC	100	100	100	102	101

2、T+1 时刻，PA 进程用完时间片放弃 CPU。何时，PA 进程会再次得到运行机会？简述 T+1 时刻系统怎样保护 PA 进程的用户态 CPU 执行现场，下次再运行时系统如何恢复 PA 进程的用户态 CPU 执行现场。

(1) T+4 时刻 PA 进程会再次得到运行机会。如表所示，T+1 时 PB 运行，T+2 时 PC 运行，T+3 时 PD 运行，T+4 时 PA 的优先级数值恢复到 100，低于 PB 和 PC、PD，重新获得 CPU。

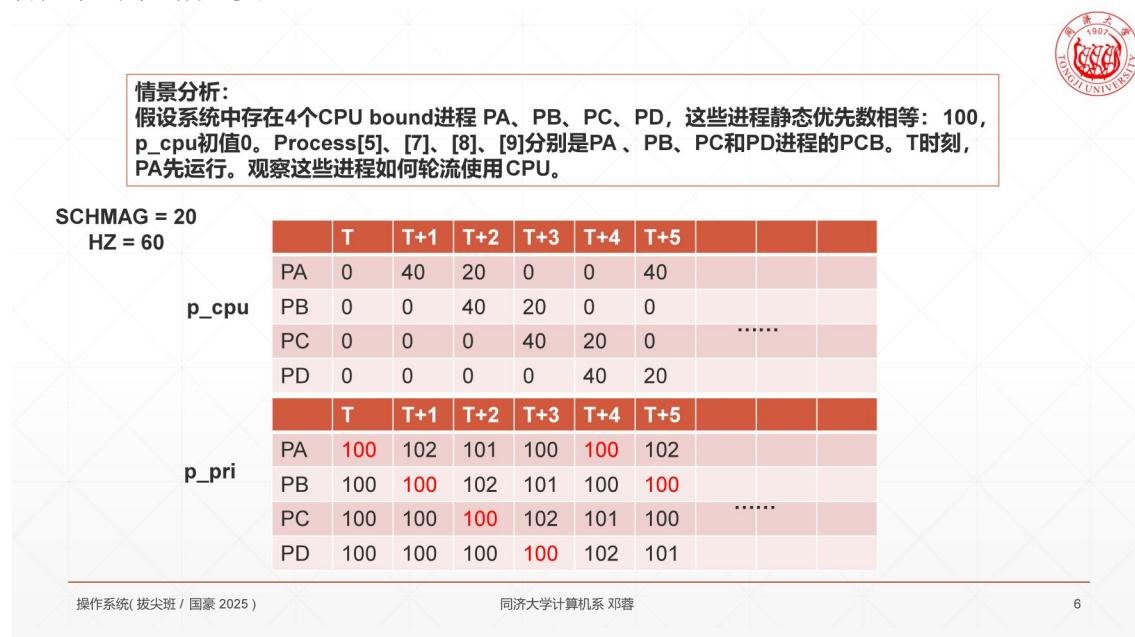
(2) 在 T+1 时刻发生时钟中断进行进程切换时，系统分三步保护现场：

- 硬件自动保存（栈切换）：CPU 响应中断，从用户态切换到核心态，自动将 PA 进程用户态的 SS, ESP, EFLAGS, CS, EIP 压入 PA 进程的核心栈。
- 中断入口保存（汇编指令）：时钟中断处理程序的入口代码执行 push 指令，将通用寄存器的值压入 PA 进程的核心栈。
- 进程切换保存：调度函数 Swtch() 将当前的核心栈指针（ESP）保存到 PA 进程的 user 结构的 u_rsav 数组中。

(3) 当在 T+3 时刻调度器决定让 PA 运行时:

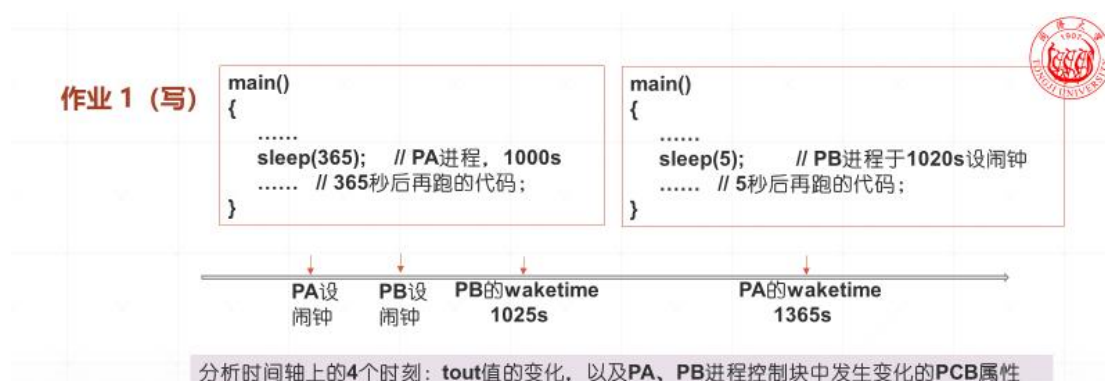
- 恢复核心栈指针: Swtch() 函数从 PA 进程 user 结构的 u_rsav 数组中读出之前保存的核心栈指针, 加载到 CPU 的 ESP 寄存器中 (此时 CPU 指向了 PA 的核心栈)。
- 恢复通用寄存器: 执行 pop 指令 (中断返回前的汇编代码), 从核心栈中弹出 T+1 时刻保存的通用寄存器的值。
- 中断返回 (IRET): 执行 IRET 指令, 硬件自动从核心栈中弹出 EIP, CS, EFLAGS, ESP, SS, 从而将 CPU 切换回用户态, 并跳转到 PA 进程此前被中断的指令处继续执行。

答, 第2小问请参考以下PPT



期末总复习: 这是Unix系统, 应用程序轮流使用CPU的方式。

Part 2、定时器服务



分析 1025 时刻 (1) 系统调度操作 (2) Sleep 系统调用下半部对 tout 变量的维护。

PA执行sleep函数, 用了多久才返回?

tout: 内核全局变量, 记录所有睡眠进程中最早的唤醒时间。

waketime: 进程自己计算的唤醒时间点 (当前时间 + sleep秒数)。

clock中断: 每秒检测 time == tout, 如果相等, 调用 WakeUpAll(&tout)。

(1) 系统调度操作

当系统时钟滴答走到 1025s 时：

- 中断判断：Clock 中断处理程序发现 `Time::time (1025) == Time::tout (1025)`。
- 唤醒操作：系统调用 `procMgr.WakeupAll(&tout)`。
- 进程状态改变：
由于 PA 和 PB 的 `p_wchan` 都是 `&tout`，两个进程都会被唤醒。
PA 和 PB 的状态从 `SSLEEP` 变为 `SRUN` (就绪)。
它们被放入就绪队列，等待调度器调度。
- CPU竞争：
假设 PB 先被调度运行。
PB 运行：从 `Sleep` 返回，检查循环条件 `while (wakeTime > Time::time)`，即 `1025 > 1025` 为假。PB 退出系统调用，返回用户态继续执行后续代码。
PA 运行：轮到 PA 执行时，它从 `Sleep` 函数内部苏醒。

(2) Sleep系统调用下半部对 tout 变量的维护

当 PA 被唤醒并获得CPU后：

- 循环检查：PA 继续执行 `Sys_Sslep` 中的代码。它检查 `while` 循环条件：
PA 的 `wakeTime` 是 1365。
当前系统时间 `Time::time` 是 1025。
`1365 > 1025` 成立，PA 发现自己还没睡够。
- tout 维护逻辑：
代码逻辑：`if ( Time::tout <= Time::time || Time::tout > wakeTime )`
此时 `tout` 的值仍是 1025 (刚刚到期的值)。
判断：`1025 <= 1025` 成立。
更新动作：`Time::tout = wakeTime`。即 `tout` 被更新为 1365。
再次入睡：PA 再次调用 `Sleep(&tout, ...)`，让出CPU，状态变为 `SSLEEP`。

(3) PA执行sleep函数，用了多久才返回？

PA 在 1000s 进入内核。

PA 在 1025s 短暂醒来，但发现时间未到，仍在内核态继续睡。

直到 1365s 时，`tout` (此时是1365) 再次到期，PA 再次被唤醒。

此时 `1365 > 1365` 为假，循环结束。

结论：PA 实际上经过了 365秒 后才真正返回用户态执行

期末总复习：以此题为背景，梳理系统调用执行细节。

1、怎样 保存恢复用户态现场；入睡、被唤醒后如何保存、恢复进程系统调用执行现场

(1) 用户态现场 (User Context) - 发生中断/系统调用瞬间

- 保存：
硬件自动保存：当执行 `int 0x80` (系统调用) 或发生时钟中断时，CPU特权级切换 (用户态->核心态)，硬件自动将用户态的 `SS, ESP, EFLAGS, CS, EIP` 压入核心栈。
软件保存：系统调用入口程序 (汇编代码) 使用 `push` 指令将通用寄存器 (`EAX, EBX, ECX, EDX...`) 压入核心栈。
- 恢复：
系统调用返回前，执行 `pop` 指令恢复通用寄存器。
执行 `IRET` 指令，硬件自动从核心栈弹出 `SS, ESP, EFLAGS, CS, EIP`，CPU切回用户态，指针回到用户程序断点处。

(2) 系统调用执行现场 (Kernel Context) - 进程切换 (入睡/唤醒)

这是在内核态进行进程切换时的保存 (即 `Swrch()` 函数的工作)：

- 保存 (入睡时)：

当 PA 在内核执行 sleep 并调用 Swtch() 让出 CPU 时。

Swtch() 会将当前的核心栈指针 (ESP)、EBP 等寄存器保存到 PA 的 user 结构的 u_rsav 数组中。

此时核心栈里已经保存了上面的“用户态现场”以及 sleep 函数的调用栈帧。保存了 ESP 就等于保存了整个执行路径。

- 恢复 (被唤醒后):

当调度器决定再次运行 PA 时, 执行 Swtch()。

从 PA user 结构的 u_rsav 数组中读出之前保存的 ESP, 加载回 CPU 的 ESP 寄存器。CPU 顺着栈帧执行 ret, 就像从刚才的 Swtch() 函数调用返回一样, 回到了 sleep 函数内部继续执行 while 判断。

2、并发的2个系统调用怎样比翼双飞

- 交替执行:

PA 和 PB 并不是在物理上“同时”执行 (单核CPU下), 而是交替使用 CPU。

- 宏观并行, 微观串行:

PA 进入内核执行 sleep(365) -> 遇到阻塞 -> 调用 Swtch() 保存现场到 PCB -> 放弃CPU。CPU 空闲, 调度器选择 PB。PB 进入内核执行 sleep(5) -> 遇到阻塞 -> 调用 Swtch() 保存现场到 PCB -> 放弃CPU。CPU 空闲, 执行 0# 进程或 Idle 循环。1025s 时钟中断 -> 唤醒 PA 和 PB。调度器先恢复 PB 的现场 (从 user 读 ESP), PB 跑完剩余内核代码 -> 返回用户态。调度器再恢复 PA 的现场 (从 user 读 ESP), PA 发现时间没到 -> 再次保存现场放弃 CPU。

所以, PA 和 PB 实际上是交替使用 CPU 执行 sleep 的上半段 (设置 tout) 和下半段 (检查 waketime)。通过全局变量 tout 和唤醒机制, 内核协调了它们的不同苏醒时间, 实现了逻辑上的并发计时。

- 关键: 内核栈隔离。每个进程都有自己独立的核心栈。PA 调用 sleep 的局部变量和返回地址存在 PA 的核心栈里; PB 的存在 PB 的核心栈里。互不干扰, 从而实现了“并发”。

Part 3、综合题

全嵌套中断处理模式。低优先级中断处理程序运行时, 系统响应高优先级中断处理请求。已知, 时钟中断优先级高于磁盘中断优先级。假设: 900s, PA 进程执行 sleep (100) 入睡。998s, PB 进程执行 read 系统调用, 读磁盘文件。1000s, 现运行进程 PX 正在执行应用程序。PA 设置的闹钟到期、PB 读取的磁盘文件数据 IO 结束。分析 1000s, 系统详细的调度过程。分两种情况考虑:

1、先响应磁盘中断

(1) 进入磁盘中断:

运行进程 PX 被磁盘中断打断。系统保存 PX 的用户态现场 (SS, ESP, EFLAGS, CS, EIP) 到核心栈。PX 进入核心态, 开始执行磁盘中断处理程序。

(2) 中断嵌套:

在执行磁盘中断处理程序期间, 1000s 的时钟中断信号到达。

由于时钟优先级 > 磁盘优先级, 且支持全嵌套, CPU 暂停磁盘中断处理程序的执行。系统保存磁盘中断处理程序的核心态现场到核心栈。PX 转而执行时钟中断处理程序。

(3) 时钟中断处理:

更新时间: 系统时间更新为 1000s。

唤醒 PA：检测到 PA 的定时器到期，唤醒 PA（状态变为 SRUN）。

检查调度：发现中断前的“先前态”是核心态（被中断的是磁盘中断处理程序），为了不干扰正在进行的硬件中断处理，时钟中断处理程序不进行调度（不调用 Swtch），仅执行 IRET 返回。

(4) 恢复磁盘中断：

CPU 回到磁盘中断处理程序继续执行。

唤醒 PB：磁盘 IO 操作完成，唤醒 PB（状态变为 SRUN）。

置调度标志：由于 PB（通常作为 IO 完成进程）和 PA 都已就绪，且优先级通常高于 CPU 密集型的 PX，系统置上 runrun 调度标志。

(5) 磁盘中断返回与调度：

磁盘中断处理程序执行完毕。

在准备返回用户态前，检查 runrun 标志，发现需要调度。

执行 Swtch()：保存 PX 的上下文（将 ESP 保存在 u.u_rsav 中），根据优先级选择最高优先级的就绪进程（通常是刚刚完成 IO 的 PB）。

PB 上台运行：恢复新进程的现场，系统进入新进程的用户态执行。

2、先响应时钟中断

(1) 进入时钟中断：

运行进程 PX 被时钟中断打断。系统保存 PX 的用户态现场到核心栈。PX 进入核心态，开始执行时钟中断处理程序。

(2) 时钟中断处理：

更新时间：系统时间更新为 1000s。

唤醒 PA：检测到 PA 定时器到期，唤醒 PA（状态变为 SRUN）。

中断返回：时钟中断处理完毕，系统执行 IRET，返回用户态。

(3) 响应磁盘中断：

返回用户态后，CPU 立即响应挂起的磁盘中断。系统再次保存 PX 的用户态现场到核心栈。PX 进入核心态，执行磁盘中断处理程序。

(4) 磁盘中断处理：

处理 IO：处理完毕磁盘数据。

唤醒 PB：唤醒 PB（状态变为 SRUN）。

置调度标志：因为有高优先级的 PA 和 PB 就绪，置 runrun 标志。

(5) 磁盘中断返回与调度：

磁盘中断处理程序准备返回用户态。

检查 runrun 标志，发现需要调度。

执行 Swtch()：调用 Swtch() 函数，保存 PX 的当前用户态栈指针（ESP）到 PX 的 user 结构（u.u_rsav）中。

进程切换：调度器从就绪队列中选择优先级最高的进程（一般是 PB，因为是 IO）。

PB（或 PA）上台运行：恢复其现场，开始执行应用程序。

Part 4、盘交换区

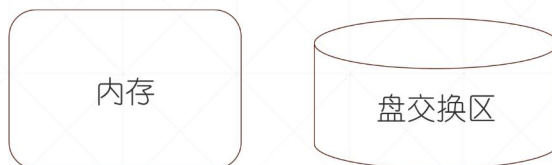
一、定义

盘交换区是主磁盘上的一块连续区域，用来容纳内存装不下的进程图像。

二、盘交换区的使用策略



二、系统实现



可执行存储器管理策略：

- (1) 内存可以容纳全部进程图像时，盘交换区不用。
 - (2) 内存不足以容纳全部进程图像时，最优布局：SWAIT放盘交换区，腾空间给SRUN和SSLEEP。
 - (3) 若内存不足以容纳全部SRUN和SSLEEP，这些进程轮流使用内存资源。
- 中级调度规则，p_time FIFO。优先换入盘交换区上驻留最久的SRUN进程；必要时，换出最先进入内存的进程。

三、情景分析，例题与习题

背景知识



3、SWAIT 入睡，唤醒 0# 进程，换出自己的进程图像

```
void Process::Sleep(unsigned long chan, int pri)
{
    if (pri > 0)
    {
        .....
        if (procMgr.RunIn != 0)
        {
            procMgr.RunIn = 0;
            procMgr.WakeupAll((unsigned long)&procMgr.RunIn);
        }
        .....
    }
}
```

PB唤醒0#进程后入睡放弃CPU。
0#进程上台，换出PB、换入PA（假设PB尺寸大于PA）。
0#进程入睡，PA上台运行。

```
Sched ( )
{
    goto loop;

Sloop:
    RunIn++, Sleep(&RunIn);
    ↓
loop: // 换入
    寻找盘交换区p_time最大的就绪进程 PA,
    其驻留时长
    if (不存在) // 等待换入任务
        RunOut++; Sleep(&RunOut); go
    else
        为PA分配内存;
        成功, goto found2
```



4、中断处理程序唤醒盘交换区上的进程 PD

```
void Process::SetRun()
{
    this->p_wchan = 0;
    this->p_stat = Process::SRUN;

    if ( this->p_pri < procMgr.CurPri )
        procMgr.RunRun++;

    if ( 0 != procMgr.RunOut && (this->p_flag & Process::SLOAD) == 0 )
    {
        procMgr.RunOut = 0;
        procMgr.WakeupAll((unsigned long)&procMgr.RunOut);
    }
}
```

```
void ProcessManager::WakeupAll(unsigned long chan)
{
    /* 唤醒系统中所有因chan而进入睡眠的进程 */
    for(int i = 0; i < ProcessManager::NPROC; i++)
        if( this->process[i].IsSleepOn(chan) )
            this->process[i].SetRun();
}
```

现运行进程执行中断处理程序唤醒 PD和0#进程，中断返回时被剥夺，放弃CPU。0#进程上台（假设内存有足够空间容纳PD），换入PD后入睡放弃CPU。PD上台运行执行系统调用下半段。。。

例题： 假设某UNIX 系统中只存在4 个进程，分别是：睡眠中的0#进程，正在CPU 上执行的pa 进程，内存中就绪状态的pb 进程（CPU bound，不会IO），盘交换区上低优先级睡眠的pc 进程。已知，当前时刻 $T=1000.5\text{ s}$ ，内存空间已满。请尽量详细地分析以下时刻系统中与进程调度和中断相关的行为，并修改下表中的相关字段。

- (1) 1000.5 s，现运行进程 pa 执行 read（读文件）系统调用
- (2) 1001.5 s，pc 等待的 IO 操作完成

序号	占用空间	优先数	状态	位置	p_time
0#	-	-100	高睡 (RunOut)	SLOAD/SSYS	-
pa	100K	103	执行	SLOAD	2
pb	10K	101	就绪	SLOAD	2
pc	60K	90	低睡	~SLOAD	3

解：

- (1) 1000.5 s，现运行进程 pa 执行 read（读文件）系统调用

1000.5s，现运行进程pa执行read（读文件）系统调用，高优先权入睡（p_stat = SSLEEP, p_pri = -50），放弃CPU。随后，系统选中pb进程，因为它是内存中就绪的唯一进程。表格修改如下：

序号	占用空间	优先数	状态	位置	p_time
0#	-	-100	高睡 (RunOut)	SLOAD/SSYS	-
pa	100K	-50	高睡 SSLEEP	SLOAD	2
pb	10K	101	执行	SLOAD	2
pc	60K	90	低睡	~SLOAD	3

1001.0s，现运行进程 pb 执行整数秒时钟中断处理程序，会++所有进程的

p_time。修正用户态进程的优先数，pb此轮执行了 0.5s，p_cpu增加30，整数秒时减20。总的效果：p_cpu加了10。整除16后，p_pri可能是102，也可能是101。表格修改如下：

序号	占用空间	优先数	状态	位置	p_time
0#	-	-100	高睡 (RunOut)	SLOAD/SSYS	-
pa	100K	-50	高睡 SSLEEP	SLOAD	3
pb	10K	101/102	执行	SLOAD	3
pc	60K	90	低睡	~SLOAD	4

(2) 1001.5 s, pc 等待的 IO 操作完成

1001.5 s。

- 现运行进程 pb 执行中断处理程序，唤醒 pc，同时唤醒 0#进程。
- 中断处理程序执行完毕后，pb 将 CPU 让给 0#进程执行 sched () 。
 - 为盘交换区上的 pc 进程分配内存空间，未果。
 - 找内存中低睡 (SWAIT) 或 SSTOP (暂停) 的进程，未果。
 - pc, p_time==4s，执行高代价对换操作。淘汰内存中的 pa 进程。具体而言，0#进程换出 pa、换入 pc (放在 pa 原先占据的内存区域) *。
- 完成对换操作后，0#进程 sleep(&RunOut,-100)入睡，将 CPU 让给内存中就绪的 pc 进程。pc 执行系统调用后半部。p-time 清 0。

表格修改如下：

序号	占用空间	优先数	状态	位置	p_time
0#	-	-100	高睡 (RunOut)	SLOAD/SSYS	-
pa	100K	-50	高睡 SSLEEP	~SLOAD	0
pb	10K	101/102	就绪 SRUN	SLOAD	3
pc	60K	90	执行 SRUN	SLOAD	0

- 系统调用执行完毕后，pc 和 pb 时间片轮转，轮流执行应用程序。

表格修改如下：

序号	占用空间	优先数	状态	位置	p_time
0#	-	-100	高睡 (RunOut)	SLOAD/SSYS	-
pa	100K	-50	高睡 SSLEEP	~SLOAD	0
pb	10K	101/102	SRUN	SLOAD	3
pc	60K	100+	SRUN	SLOAD	0

pb、pc 都有可能是现运行进程。

*处的细节，不要求掌握。就是换入、换出过程中，0#进程会睡眠等待IO结束的，此时pb使用CPU，是现运行进程。一旦IO完成，pb立即将CPU让给0#进程，因为它优先级最高。

习题： T0时刻，某 UNIX V6++系统进程状态如下表所示。内存空间已满，除图示空间外，其余空间不可用。请尽量详细地分析以下时刻系统中与进程调度和对换操作（swap in, swap out）相关的行为，并修改下表中的相关字段。

序号	占用空间	状态	位置	内存起始物理地址
0#	-	高睡（RunOut）	SLOAD/SSYS	0x003ff000(4M 核心空间最后一个物理页框)
p1	40K	低睡	SLOAD	0x00408000
p2	30K	执行	SLOAD	0x00430000
p3	30K	低睡	~SLOAD	0x00450000

注释：高亮部分，盘交换区上的进程，内存起始地址字段答题不需要。是上一次在内存，为进程分配的内存空间的首地址，留着它有用的。

（1）T0时刻（非整数秒），现运行进程 p2 执行 read 系统调用读磁盘文件（磁盘高速缓存中没有 p2 需要的文件数据）

- 现运行进程 p2 执行 read 系统调用读取磁盘文件。磁盘高速缓存中没有 p2 需要的文件数据，必须启动物理磁盘 I/O。p2 启动磁盘驱动器。p2 执行 sleep() 等待 I/O 完成，p2 进入高优先级睡眠（SSLEEP/高睡）状态，放弃 CPU。

- 调度：

p1 处于低睡状态（SWAIT）。

p3 处于低睡状态且在盘交换区。

内存中没有就绪（SRUN）进程。系统进入 Idle（空闲）状态，0# 进程被短暂唤醒查看无事可做后继续睡眠。

序号	占用空间	状态	位置	内存起始物理地址
0#	-	高睡（RunOut）	SLOAD/SSYS	0x003ff000(4M 核心空间最后一个物理页框)
p1	40K	低睡	SLOAD	0x00408000
p2	30K	高睡（SSLEEP）	SLOAD	0x00430000
p3	30K	低睡	~SLOAD	0x00450000

（2）T1（T1=T0+1 秒）时刻，已完成 read 系统调用的 p2 进程运行在用户态。p3 等待的 I/O 操作完成。

中断处理程序调用 wakeup 唤醒 p3。p3 状态变为就绪（SRUN）。由于 p3 位于盘交换区（~SLOAD），wakeup 发现就绪进程在磁盘上，于是同时唤醒 0# 进程（Swapper）。

0# 试图将 p3 换入内存（Swap In）。

内存判断：p3 需要 30K 空间。内存空间已满。0# 必须在内存中寻找一个进程换出以腾出空间。

内存中有：p1（低睡 SWAIT）、p2（就绪 SRUN）。

优先换出处于低优先级睡眠（SWAIT/低睡）的进程。选中 p1 进行换出。

执行对换：

将 p1 的图像写入盘交换区。p1 位置变为 ~SLOAD。

释放 p1 占用的内存（起始地址 0x00408000，大小 40K）。

将 p3 从盘交换区读入内存。由于 40K（释放的）> 30K（需要的），空间足够。
p3 位置变为 SLOAD，其内存起始地址变为原 p1 的地址 0x00408000。

序号	占用空间	状态	位置	内存起始物理地址
0#	-	高睡（RunOut）	SLOAD/SSYS	0x003ff000(4M 核 心 空 间最后一个物理页框)
p1	40K	低睡	~SLOAD	
p2	30K	执行/就绪	SLOAD	0x00430000
p3	30K	就绪/执行	SLOAD	0x00408000